Scripts Documentation

High5

June 28, 2025

Contents

1 merge_data.py

Purpose: Merges multiple datasets into a single dataframe.

```
# Merging new dataset provided by teknofest
import polars as pl

dl_data = pl.read_excel("data/5g-field-data/5G_DL.xlsx", sheet_name="Series Formatted Data")
dl_test_data = pl.read_excel("data/5g-field-data/test-data/5G_DL.xlsx", sheet_name="Series Formatted Data")

dl_test_data = dl_test_data.with_columns((pl.col("Message")+ dl_data["Message"][-1] + 1))

for col in dl_data.columns:
    if col not in dl_test_data.columns:
        dl_data = dl_data.drop(col)

for col in dl_test_data.columns:
    if col not in dl_data.columns:
        dl_test_data = dl_test_data.drop(col)

cat_cols = [
    "NR_UE_PCI_0",
    "NR_UE_Nbr_PCI_0",
    "NR_UE_Nbr_PCI_1",
    "NR_UE_Nbr_PCI_2",
    "NR_UE_Nbr_PCI_3",
    "NR_UE_RI_DL_0",
    "NR_UE_Modulation_Avg_DL_0",
    "NR_UE_MCS_DL_0",
    "NR_UE_CCE_AggregationLev_0",
    "NR_RRC_MsgType",
    "NR_UE_RRCReEst_EndResult",
    "NAS_5GS_MM_MessageType"
]

merged_data = pl.concat([dl_data,dl_test_data],how='vertical_relaxed')

merged_data = merged_data.drop("Technology_Mode")

for col in merged_data.columns:
    if col == "Time":
        continue
    elif "_PCI_" in col:
        merged_data = merged_data.with_columns(
            pl.col(col).cast(pl.Float64, strict=True)
        )
    elif (merged_data[col].dtype == pl.String) and col not in cat_cols:
        merged_data = merged_data.with_columns(
            pl.col(col).cast(pl.Float64, strict=True)
        )

merged_data.write_parquet("data/parquet/dl.parquet")+"
```

2 handle_nulls.py

Purpose: Imputation of null values.

```
import polars as pl
from calculations_util.distance import pathloss_to_distance

cell_data = pl.read_parquet("data/parquet/cell_info.parquet")
df = pl.read_parquet("data/parquet/dl.parquet").sort("Message")


df_flags = (
    df.select(
```

```
        "Time",
        "NR_UE_RRCHOAttempt",
        "NR_UE_RRCHOOK",
        "NR_UE_RACH_OK",
        "NR_UE_RACH_Attempt",
    )
    .group_by("Time")
    .agg(
        [
            (pl.col("NR_UE_RACH_Attempt") == 1)
            .any()
            .cast(pl.Int8)
            .alias("rach_attempt"),
            (pl.col("NR_UE_RACH_OK") == 1).any().cast(pl.Int8).alias("rach_ok"),
            (pl.col("NR_UE_RRCHOAttempt") == 1).any().cast(pl.Int8).alias("ho_attempt"),
            (pl.col("NR_UE_RRCHOOK") == 1).any().cast(pl.Int8).alias("ho_ok"),
        ]
    )
)

fill_cols = [
    "NR_UE_Timing_Advance",
    "NR_UE_Pathloss_DL_0",
    "NR_UE_Throughput_PDCP_DL",
    "App_Throughput_DL",
    "NR_UE_NACK_Rate_DL_0",
    "NR_UE_Ack_As_Nack_DL_0",
    "NR_UE_MCS_DL_0",
    "NR_UE_RB_Num_DL_0",
    "NR_UE_Modulation_Avg_DL_0",
    "NR_UE_RI_DL_0",
    "NR_UE_BLER_DL_0",
    "NR_UE_CCE_AggregationLev_0",
    "NR_UE_Power_Tx_PUSCH_0",
    "NR_UE_Power_Tx_PRACH_0",
    "NR_UE_NACK_Rate_UL_0",
]

df = df.with_columns([
    pl.col(col).forward_fill().backward_fill().over("Time").alias(col)
    for col in fill_cols
])

df = df.filter(
    pl.col("NR_RRC_MsgType").is_null()
).drop(
    [
        "NR_UE_RRCHOAttempt",
        "NR_UE_RRCHOOK",
        "NR_UE_RACH_OK",
        "NR_UE_RACH_Attempt",
        "NR_RRC_MsgType",
    ]
)

group_cols = [
    "Longitude",
    "Latitude",
    "Time",
    "NR_UE_PCI_0",
    "NR_UE_RSRP_0",
    "NR_UE_RSRQ_0",
    "NR_UE_SINR_0",
    "NR_UE_Nbr_PCI_0",
    "NR_UE_Nbr_PCI_1",
    "NR_UE_Nbr_PCI_2",
    "NR_UE_Nbr_PCI_3",
    "NR_UE_Nbr_RSRP_0",
    "NR_UE_Nbr_RSRP_1",
    "NR_UE_Nbr_RSRP_2",
    "NR_UE_Nbr_RSRP_3",
    "NR_UE_Nbr_RSRQ_0",
    "NR_UE_Nbr_RSRQ_1",
```

```
    "NR_UE_Nbr_RSRQ_2",
    "NR_UE_Nbr_RSRQ_3",
]

df = (
    df.group_by(group_cols)
    .agg(
        pl.col("Message").max(), *[pl.col(col).first().alias(col) for col in fill_cols]
    )
    .join(df_flags, on="Time")
    .drop("Time")
    .filter(pl.col("NR_UE_PCI_0").is_in(list(cell_data["PCI"])),
            pl.col("NR_UE_PCI_0").is_not_null(),
            pl.col("NR_UE_RSRP_0").is_not_null())
)

# replacing with bit/symbol

df = df.with_columns(
    [
        pl.when(pl.col("NR_UE_Modulation_Avg_DL_0") == "QPSK")
        .then(4)
        .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "16QAM")
        .then(16)
        .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "64QAM")
        .then(64)
        .when(pl.col("NR_UE_Modulation_Avg_DL_0") == "256QAM")
        .then(256)
        .otherwise(None)
        .alias("modulation_avg_encoded"),
    ]
)

df = df.with_columns(
    [pl.col("modulation_avg_encoded").log(base=2).alias("modulation_avg_log2_bits")]
)

df = df.with_columns(
    [
        pl.when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_1")
        .then(1)
        .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_2")
        .then(2)
        .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_4")
        .then(4)
        .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_8")
        .then(8)
        .when(pl.col("NR_UE_CCE_AggregationLev_0") == "LEVEL_16")
        .then(16)
        .otherwise(None)
        .alias("cce_level_encoded")
    ]
)

df = df.with_columns(
    [
        pl.col("NR_UE_MCS_DL_0")
        .str.extract(r"^(\d+)", group_index=1)  # extracting mcs index at the beggining
        .cast(pl.Int64)
        .alias("mcs_index")
    ]
)

df = df.with_columns(
    pathloss_to_distance(pl.col("NR_UE_Pathloss_DL_0")).alias("pl_distance")
)

df = df.with_columns(
    [
        (pl.col("NR_UE_Pathloss_DL_0") / pl.col("pl_distance")).alias("pl_per_meter"),
        (pl.col("pl_distance").log1p().alias("log1p_pl_distance")),
        (pl.col("App_Throughput_DL") / pl.col("NR_UE_RB_Num_DL_0")).alias(
            "spectral_efficiency"
```

```
        ),
        (pl.col("modulation_avg_log2_bits") / (pl.col("NR_UE_RB_Num_DL_0"))).alias(
            "modulation_efficiency"
        ),
        (pl.col("NR_UE_Throughput_PDCP_DL") / pl.col("pl_distance")).alias(
            "throughput_per_meter"
        ),
        (pl.col("NR_UE_SINR_0") / (pl.col("pl_distance"))).alias("sinr_per_meter"),
        (pl.col("NR_UE_SINR_0") / (pl.col("NR_UE_RSRP_0"))).alias("sinr_over_rsrp"),
        (pl.col("NR_UE_RSRP_0") - pl.col("NR_UE_Nbr_RSRP_0")).alias("rsrp_delta"),
        (pl.col("NR_UE_RSRQ_0") - pl.col("NR_UE_Nbr_RSRQ_0")).alias("rsrq_delta"),
        (pl.col("NR_UE_RSRP_0") / pl.col("NR_UE_Pathloss_DL_0")).alias(
            "signal_loss_ratio"
        ),
    ]
)

df.write_parquet("data/parquet/processed/dl.parquet")
```

3 add_cluster_info.py

Purpose: Adds cluster information to the dataset for segmentation purposes.

```

# create folds with agglomerative clustering and save folds as a pickle file.
import os
import pickle
import polars as pl
import geopandas as gpd
from sklearn.metrics.pairwise import haversine_distances
from sklearn.cluster import AgglomerativeClustering
from pyproj import Geod

geod = Geod(ellps="WGS84")

df = pl.read_parquet("data/parquet/processed/dl.parquet")
df_clean = df.drop_nulls(subset=["Longitude", "Latitude"]).select(
    pl.col("Longitude").radians(), pl.col("Latitude").radians(), "Message"
)

n_clusters = 10
X = df_clean[["Latitude", "Longitude"]].to_numpy()

agg = AgglomerativeClustering(
    n_clusters=n_clusters, metric=haversine_distances, linkage="complete"
)
agg.fit_predict(X)

complete = df.join(
    df_clean.with_columns(pl.Series(name="cluster", values=agg.fit_predict(X))).select(
        "cluster", "Message"
    ),
    on="Message",
    how="left",
)

complete.write_parquet("data/parquet/processed/dl.with_cluster.parquet")

cluster_centers = {}

for cluster in range(n_clusters):
    df_filtered = complete.select("Longitude", "Latitude", "cluster").filter(
        (pl.col("cluster") == cluster)
    )

    geometry = gpd.points_from_xy(
        df_filtered["Longitude"], df_filtered["Latitude"], crs="EPSG:4326"
    )
    gdf = gpd.GeoDataFrame(geometry=geometry)
    gdf = gdf.to_crs(
```

```
        "EPSG:32635"
    )  # projecting to local UTM zone for accurate centroid calculation.
    centroid = gdf.union_all().centroid
    centroid_gdf = gpd.GeoDataFrame(geometry=[centroid], crs="EPSG:32635").to_crs(
        "EPSG:4326"
    )  # converting centroid back
    centroid = centroid_gdf.geometry.x.iloc[0], centroid_gdf.geometry.y.iloc[0]
    cluster_centers[cluster] = centroid

# print(f"clusters and cluster centroids:\n{cluster_centers}")

distance_dict = {}
distance_threshold = 700

for i in range(len(cluster_centers)):
    lon1, lat1 = cluster_centers[i]
    for j in range(i + 1, len(cluster_centers)):
        lon2, lat2 = cluster_centers[j]
        _, _, distance = geod.inv(lon1, lat1, lon2, lat2)
        if distance_threshold < distance:
            distance_dict[(i, j)] = distance

# print(f"distance between fold centers:\n{distance_dict}")

folds = {}

for pairs in distance_dict:
    if pairs[0] not in folds:
        folds[pairs[0]] = [pairs[1]]
    else:
        folds[pairs[0]].append(pairs[1])
    if pairs[1] not in folds:
        folds[pairs[1]] = [pairs[0]]
    else:
        folds[pairs[1]].append(pairs[0])


# deleting folds where test fold size > train fold size

del_folds = False
if del_folds:
    for fold in folds.copy():
        test_cluster = fold
        train_clusters = folds[fold]

        test_size = len(complete.filter(pl.col("cluster") == test_cluster))
        train_size = len(complete.filter(pl.col("cluster").is_in(train_clusters)))

        if test_size > train_size:
            del folds[fold]

os.makedirs("data/folds", exist_ok=True)

with open("data/folds/folds.pkl", "wb") as f:
    pickle.dump(folds, f)

# print(f"folds:\n{folds}")

# plot clusters if you want to check
"""
df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet")

plt.figure(figsize=(10, 6))
scatter = plt.scatter(
    df["Longitude"],
    df["Latitude"],
    c=df["cluster"],
    cmap="tab10",
    s=10,
)

for cluster_id, (lon, lat) in cluster_centers.items():
    plt.scatter(lon, lat, c="red", marker="o", s=100)
```

```
    plt.text(lon, lat, str(cluster_id), color="black", fontsize=15, ha="center")

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.grid(True)
plt.show()
"""
```

4 add_cell_info_v2.py

Purpose: Adds base station (cell) information to enrich the dataset.

```
import polars as pl
import geopandas as gpd
from pyproj import Geod

geod = Geod(ellps="WGS84")

cell_data = pl.read_parquet("data/parquet/cell_info.parquet")
df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet")
coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
intersections_2 = gpd.read_parquet(
    "data/parquet/coverages/coverages_intersect_2.parquet"
)

coverages = pl.DataFrame(
    {
        "centroid_longitude": coverages.clipped_coverage_centroid.x,
        "centroid_latitude": coverages.clipped_coverage_centroid.y,
        "PCI": coverages["PCI"],
    }
)

intersections_2 = pl.DataFrame(
    {
        "sorted_cell_pairs": intersections_2["sorted_cell_pairs"],
        "intersection_2_convex_hull": intersections_2["convex_hull"],
        "intersection_2_area_m2": intersections_2["area_m2"],
        "intersection_2_centroid_lon": intersections_2["centroid"].x,
        "intersection_2_centroid_lat": intersections_2["centroid"].y,
    }
)

pci_cols = [col for col in df.columns if "PCI" in col]

for i, pci_col in enumerate(pci_cols):
    df = df.join(
        cell_data.select(
            pl.col("Longitude"),
            pl.col("Latitude"),
            pl.col("Azimuth␣[  ]").radians().sin().alias(f"Azimuth_nn{i}_cell_sin"),
            pl.col("Azimuth␣[  ]").radians().cos().alias(f"Azimuth_nn{i}_cell_cos"),
            pl.when(pl.col("MIMO") == "64T64R").then(64)
            .when(pl.col("MIMO") == "32T32R").then(32)
            .otherwise(None)
            .alias(f"mimo_nn{i}_cell"),
            "PCI",
        ),
        left_on=pci_col,
        right_on="PCI",
        how="left",
        suffix=f"_nn{i}_cell",
    )

    # joining connected cell's azimuth in degrees, which is going to be used for geod
        fwd function below.
    if i == 0:
        df = df.join(
            cell_data.select(
                pl.col("Azimuth␣[  ]").alias("Azimuth_nn0_cell"),
```

```
                "PCI",
            ),
            left_on=pci_col,
            right_on="PCI",
            how="left",
        )

    df = df.join(
        coverages.select(
            pl.col("centroid_longitude").alias(f"centroid_lon_nn{i}_cell_coverage"),
            pl.col("centroid_latitude").alias(f"centroid_lat_nn{i}_cell_coverage"),
            "PCI",
        ),
        left_on=pci_col,
        right_on="PCI",
        how="left",
    )
df = (
    df.with_columns(
        pl.concat_list([pl.col("NR_UE_PCI_0"), pl.col("NR_UE_Nbr_PCI_0")])
        .list.sort()
        .alias("sorted_cell_pairs")
    )
    .join(intersections_2, how="left", on="sorted_cell_pairs")
    .drop("sorted_cell_pairs")
)

ue_lon, ue_lat, _ = geod.fwd(
    lons=df["Longitude_nn0_cell"],
    lats=df["Latitude_nn0_cell"],
    az=df["Azimuth_nn0_cell"],
    #dist=df["distance"], old column name
    dist=df["pl_distance"],
)

df = df.with_columns(
    [pl.Series("pl_predicted_longitude", ue_lon), pl.Series("pl_predicted_latitude",
        ue_lat)
    ]
)


"""
old df column names

df = df.with_columns([
    (pl.col("spectral_efficiency") / pl.col("mimo_nn0_cell")).alias("
        efficiency_per_antenna"),
    (pl.col("m_NR_UE_SINR_0") / pl.col("mimo_nn0_cell")).alias("sinr_per_antenna"),
    (pl.col("mimo_nn0_cell") / (pl.col("distance") + 1)).alias("mimo_distance_ratio"),
    (pl.col("mimo_nn0_cell") * pl.col("m_App_Throughput_DL")).alias("
        mimo_throughput_potential"),
    (pl.col("signal_loss_ratio") * pl.col("mimo_nn0_cell")).alias("scaled_loss_ratio"),
    (pl.col("Azimuth_nn0_cell_cos") * -pl.col("m_NR_UE_RSRP_0")).alias("
        directional_rsrp_nn0_lon"),
    (pl.col("Azimuth_nn0_cell_sin") * -pl.col("m_NR_UE_RSRP_0")).alias("
        directional_rsrp_nn0_lat"),
    (pl.col("Azimuth_nn1_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_1")).alias("
        directional_rsrp_nn1_lon"),
    (pl.col("Azimuth_nn1_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_1")).alias("
        directional_rsrp_nn1_lat"),
    (pl.col("Azimuth_nn2_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_2")).alias("
        directional_rsrp_nn2_lon"),
    (pl.col("Azimuth_nn2_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_2")).alias("
        directional_rsrp_nn2_lat"),
    (pl.col("Azimuth_nn3_cell_cos") * -pl.col("m_NR_UE_Nbr_RSRP_3")).alias("
        directional_rsrp_nn3_lon"),
    (pl.col("Azimuth_nn3_cell_sin") * -pl.col("m_NR_UE_Nbr_RSRP_3")).alias("
        directional_rsrp_nn3_lat"),
])
"""

df = df.with_columns([
```

```
    (pl.col("spectral_efficiency") / pl.col("mimo_nn0_cell")).alias("efficiency_per_antenna"),
    (pl.col("NR_UE_SINR_0") / pl.col("mimo_nn0_cell")).alias("sinr_per_antenna"),
    (pl.col("mimo_nn0_cell") / (pl.col("pl_distance") + 1)).alias("mimo_distance_ratio"),
    (pl.col("mimo_nn0_cell") * pl.col("App_Throughput_DL")).alias("mimo_throughput_potential"),
    (pl.col("signal_loss_ratio") * pl.col("mimo_nn0_cell")).alias("scaled_loss_ratio"),
    (pl.col("Azimuth_nn0_cell_cos") * -pl.col("NR_UE_RSRP_0")).alias("directional_rsrp_nn0_lon"),
    (pl.col("Azimuth_nn0_cell_sin") * -pl.col("NR_UE_RSRP_0")).alias("directional_rsrp_nn0_lat"),
    (pl.col("Azimuth_nn1_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_1")).alias("directional_rsrp_nn1_lon"),
    (pl.col("Azimuth_nn1_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_1")).alias("directional_rsrp_nn1_lat"),
    (pl.col("Azimuth_nn2_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_2")).alias("directional_rsrp_nn2_lon"),
    (pl.col("Azimuth_nn2_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_2")).alias("directional_rsrp_nn2_lat"),
    (pl.col("Azimuth_nn3_cell_cos") * -pl.col("NR_UE_Nbr_RSRP_3")).alias("directional_rsrp_nn3_lon"),
    (pl.col("Azimuth_nn3_cell_sin") * -pl.col("NR_UE_Nbr_RSRP_3")).alias("directional_rsrp_nn3_lat"),
])

df.write_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")
```

5 exp_optuna_v2.py

Purpose: Runs experiments using Optuna for hyperparameter optimization.

```

import os
from pathlib import Path
import polars as pl
import pickle
from sklearn.ensemble import StackingRegressor
from sklearn.multioutput import MultiOutputRegressor
from lightgbm import LGBMRegressor
from sklearn.preprocessing import PolynomialFeatures
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
from sklearn.metrics import make_scorer
import optuna
import webbrowser
from datetime import datetime
from xgboost import XGBRegressor
from cell_util.ml import CustomCV
from calculations_util.distance import haversine_np

df = (
    pl.read_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")
    .drop_nulls(["Longitude", "Latitude"])
    .select(
        "pl_distance",
        "log1p_pl_distance",
        "pl_per_meter",
        "pl_predicted_longitude",
        "pl_predicted_latitude",
        "throughput_per_meter",
        "spectral_efficiency",
        "sinr_per_meter",
        "sinr_over_rsrp",
        "signal_loss_ratio",
        "rsrq_delta",
        "rsrp_delta",
        "scaled_loss_ratio",
        "mimo_throughput_potential",
```

```
        "mimo_distance_ratio",
        "sinr_per_antenna",
        "efficiency_per_antenna",
        "modulation_efficiency",
        "mimo_nn0_cell",
        "mimo_nn1_cell",
        "mimo_nn2_cell",
        "mimo_nn3_cell",
        "Longitude",
        "Latitude",
        "NR_UE_Pathloss_DL_0",
        "Longitude_nn0_cell",
        "Latitude_nn0_cell",
        "Longitude_nn1_cell",
        "Latitude_nn1_cell",
        "Longitude_nn2_cell",
        "Latitude_nn2_cell",
        "Longitude_nn3_cell",
        "Latitude_nn3_cell",
        "Azimuth_nn0_cell_cos",
        "Azimuth_nn0_cell_sin",
        "Azimuth_nn1_cell_cos",
        "Azimuth_nn1_cell_sin",
        "Azimuth_nn2_cell_cos",
        "Azimuth_nn2_cell_sin",
        "Azimuth_nn3_cell_cos",
        "Azimuth_nn3_cell_sin",
        "directional_rsrp_nn0_lon",
        "directional_rsrp_nn0_lat",
        "directional_rsrp_nn1_lon",
        "directional_rsrp_nn1_lat",
        "directional_rsrp_nn2_lon",
        "directional_rsrp_nn2_lat",
        "directional_rsrp_nn3_lon",
        "directional_rsrp_nn3_lat",
        "cluster",
        "NR_UE_RSRP_0",
        "NR_UE_Nbr_RSRP_0",
        "NR_UE_Nbr_RSRP_1",
        "NR_UE_Nbr_RSRP_2",
        "NR_UE_Nbr_RSRP_3",
        "NR_UE_SINR_0",
        "NR_UE_RB_Num_DL_0",
        "NR_UE_RSRQ_0",
        "NR_UE_Nbr_RSRQ_0",
        "NR_UE_Nbr_RSRQ_1",
        "NR_UE_Nbr_RSRQ_2",
        "NR_UE_Nbr_RSRQ_3",
        "NR_UE_BLER_DL_0",
        "NR_UE_Timing_Advance",
        "rach_attempt",
        "rach_ok",
        "ho_attempt",
        "ho_ok",
        "App_Throughput_DL",
        "NR_UE_Throughput_PDCP_DL",
        "NR_UE_Ack_As_Nack_DL_0",
        "NR_UE_NACK_Rate_DL_0",
        "NR_UE_Power_Tx_PUSCH_0",
        "NR_UE_Power_Tx_PRACH_0",
        "NR_UE_NACK_Rate_UL_0",
        "modulation_avg_encoded",
        "modulation_avg_log2_bits",
        "cce_level_encoded",
        "mcs_index",
        "centroid_lon_nn0_cell_coverage",
        "centroid_lat_nn0_cell_coverage",
        "centroid_lon_nn1_cell_coverage",
        "centroid_lat_nn1_cell_coverage",
        "centroid_lon_nn2_cell_coverage",
        "centroid_lat_nn2_cell_coverage",
        "centroid_lon_nn3_cell_coverage",
        "centroid_lat_nn3_cell_coverage",
```

```
        "intersection_2_centroid_lon",
        "intersection_2_centroid_lat",
    )
    .to_pandas()
)

with open("data/folds/folds.pkl", "rb") as f:
    folds = pickle.load(f)


y = df[["Latitude", "Longitude"]].to_numpy()

X = df.drop(["Latitude", "Longitude", "cluster"], axis=1).to_numpy()

scorer = make_scorer(score_func=haversine_np, greater_is_better=False)


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 100, 700)
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 0.01)
    max_depth = trial.suggest_int("max_depth", 3, 20)
    # gamma = trial.suggest_float("gamma", 0.0, 1e-6) # trials  may stuck at score -850.9125320592835 when upper bound = 1-e2 or higher
    reg_alpha = trial.suggest_float("reg_alpha", 1e-4, 0.20)
    reg_lambda = trial.suggest_float("reg_lambda", 0.0, 0.15)
    subsample = trial.suggest_float("subsample", 0.5, 1.0)
    colsample_bytree = trial.suggest_float("colsample_bytree", 0.5, 1.0)
    #colsample_bylevel = trial.suggest_float("colsample_bylevel", 0.5, 1.0)
    #colsample_bynode = trial.suggest_float("colsample_bynode", 0.5, 1.0)
    min_child_weight = trial.suggest_int("min_child_weight", 1, 60)
    grow_policy = trial.suggest_categorical("grow_policy", ["depthwise", "lossguide"])

    if grow_policy == "lossguide":
        max_leaves, max_depth = trial.suggest_int("max_leaves", 16, 512), 0
    else:
        max_leaves = 0

    xgb = XGBRegressor(
        n_estimators=n_estimators,
        learning_rate=learning_rate,
        max_depth=max_depth,
        max_leaves=max_leaves,
        reg_alpha=reg_alpha,
        reg_lambda=reg_lambda,
        subsample=subsample,
        colsample_bytree=colsample_bytree,
        min_child_weight=min_child_weight,
        grow_policy=grow_policy,
        sampling_method="uniform",
        tree_method="hist",
        n_jobs=-1,
        objective="reg:squarederror",
        seed=23,
    )

    """
    lgbm_n_estimators = trial.suggest_int("lgbm_n_estimators", 100, 600)
    lgbm_learning_rate = trial.suggest_float("lgbm_learning_rate", 1e-3, 0.01)
    lgbm_max_depth = trial.suggest_int("lgbm_max_depth", 3, 12)
    lgbm_reg_alpha = (trial.suggest_float("lgbm_reg_alpha", 0.0, 0.15),)
    lgbm_reg_lambda = (trial.suggest_float("lgbm_reg_lambda", 0.0, 0.15),)
    lgbm_importance_type = (
        trial.suggest_categorical("lgbm_importance_type", ["split", "gain"]),
    )
    lgbm_subsample = trial.suggest_float("lgbm_subsample", 0.8, 1.0)

    lgbm = LGBMRegressor(
        n_estimators=lgbm_n_estimators,
        learning_rate=lgbm_learning_rate,
        max_depth=lgbm_max_depth,
        reg_alpha=lgbm_reg_alpha,
        reg_lambda=lgbm_reg_lambda,
        subsample=lgbm_subsample,
```

```
        importance_type=lgbm_importance_type,
        objective="regression",
        n_jobs=-1,
        random_state=23,
        verbose=-1,
    )
    """

    #degree = trial.suggest_int("poly_degree", 1, 4)
    """
    meta_learner = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        RandomForestRegressor(n_estimators=100, random_state=23, n_jobs=-1),
    )

    stacking = StackingRegressor(
        estimators=[("xgb", xgb)],
        final_estimator=meta_learner,
        n_jobs=-1,
    )
    """
    model = MultiOutputRegressor(xgb)

    scores = cross_val_score(
        model, X, y, scoring=scorer, cv=CustomCV(folds, df["cluster"])
    )
    return scores.mean()


study = optuna.create_study(
    direction="maximize", sampler=optuna.samplers.RandomSampler(seed=23)
)
# study = optuna.create_study(direction="maximize", sampler=optuna.samplers.TPESampler(
    seed=23))

study.optimize(objective, n_trials=4000, n_jobs=5)
print(f"\nBest␣Parameters:\n{study.best_params}\n")


os.makedirs("optuna/plots", exist_ok=True)
os.makedirs("optuna/trial-data", exist_ok=True)

timestamp = datetime.now().strftime("%d-%m-%Y_%H-%M-%S")
study.trials_dataframe().to_csv(f"optuna/trial-data/optuna_trials_{timestamp}.csv")

fig_hist = optuna.visualization.plot_optimization_history(study)
fig_hist.write_html(f"optuna/plots/optimization_history_{timestamp}.html")

fig_imp = optuna.visualization.plot_param_importances(study)
fig_imp.write_html(f"optuna/plots/param_importances_{timestamp}.html")

fig_slice = optuna.visualization.plot_slice(
    study, params=list(study.best_trial.params.keys())
)
fig_slice.write_html(f"optuna/plots/param_slice_{timestamp}.html")

open_webbrowser = True
if open_webbrowser:
    webbrowser.open(
        Path(f"optuna/plots/optimization_history_{timestamp}.html").resolve().as_uri()
    )
    webbrowser.open(
        Path(f"optuna/plots/param_importances_{timestamp}.html").resolve().as_uri()
    )
    webbrowser.open(
        Path(f"optuna/plots/param_slice_{timestamp}.html").resolve().as_uri()
    )
```

6 do_experiment_v2.py

Purpose: Analyzes experiment results, generates feature importance and visualizations.

```
1
2  import polars as pl
3  import pickle
4  from sklearn.multioutput import MultiOutputRegressor
5  import matplotlib.pyplot as plt
6  from sklearn.model_selection import cross_val_score
7  from sklearn.metrics import make_scorer
8
9  from xgboost import XGBRegressor
10 from cell_util.ml import CustomCV
11 from calculations_util.distance import haversine_np
12
13 df = (
14     pl.read_parquet("data/parquet/processed/dl.with_cluster_cell_info.parquet")
15     .drop_nulls(["Longitude", "Latitude"])
16     .select(
17         "pl_distance",
18         "log1p_pl_distance",
19         "pl_per_meter",
20         "pl_predicted_longitude",
21         "pl_predicted_latitude",
22         "throughput_per_meter",
23         "spectral_efficiency",
24         "sinr_per_meter",
25         "sinr_over_rsrp",
26         "signal_loss_ratio",
27         "rsrq_delta",
28         "rsrp_delta",
29         "scaled_loss_ratio",
30         "mimo_throughput_potential",
31         "mimo_distance_ratio",
32         "sinr_per_antenna",
33         "efficiency_per_antenna",
34         "modulation_efficiency",
35         "mimo_nn0_cell",
36         "mimo_nn1_cell",
37         "mimo_nn2_cell",
38         "mimo_nn3_cell",
39         "Longitude",
40         "Latitude",
41         "NR_UE_Pathloss_DL_0",
42         "Longitude_nn0_cell",
43         "Latitude_nn0_cell",
44         "Longitude_nn1_cell",
45         "Latitude_nn1_cell",
46         "Longitude_nn2_cell",
47         "Latitude_nn2_cell",
48         "Longitude_nn3_cell",
49         "Latitude_nn3_cell",
50         "Azimuth_nn0_cell_cos",
51         "Azimuth_nn0_cell_sin",
52         "Azimuth_nn1_cell_cos",
53         "Azimuth_nn1_cell_sin",
54         "Azimuth_nn2_cell_cos",
55         "Azimuth_nn2_cell_sin",
56         "Azimuth_nn3_cell_cos",
57         "Azimuth_nn3_cell_sin",
58         "directional_rsrp_nn0_lon",
59         "directional_rsrp_nn0_lat",
60         "directional_rsrp_nn1_lon",
61         "directional_rsrp_nn1_lat",
62         "directional_rsrp_nn2_lon",
63         "directional_rsrp_nn2_lat",
64         "directional_rsrp_nn3_lon",
65         "directional_rsrp_nn3_lat",
66         "cluster",
67         "NR_UE_RSRP_0",
68         "NR_UE_Nbr_RSRP_0",
```

```
        "NR_UE_Nbr_RSRP_1",
        "NR_UE_Nbr_RSRP_2",
        "NR_UE_Nbr_RSRP_3",
        "NR_UE_SINR_0",
        "NR_UE_RB_Num_DL_0",
        "NR_UE_RSRQ_0",
        "NR_UE_Nbr_RSRQ_0",
        "NR_UE_Nbr_RSRQ_1",
        "NR_UE_Nbr_RSRQ_2",
        "NR_UE_Nbr_RSRQ_3",
        "NR_UE_BLER_DL_0",
        "NR_UE_Timing_Advance",
        "rach_attempt",
        "rach_ok",
        "ho_attempt",
        "ho_ok",
        "App_Throughput_DL",
        "NR_UE_Throughput_PDCP_DL",
        "NR_UE_Ack_As_Nack_DL_0",
        "NR_UE_NACK_Rate_DL_0",
        "NR_UE_Power_Tx_PUSCH_0",
        "NR_UE_Power_Tx_PRACH_0",
        "NR_UE_NACK_Rate_UL_0",
        "modulation_avg_encoded",
        "modulation_avg_log2_bits",
        "cce_level_encoded",
        "mcs_index",
        "centroid_lon_nn0_cell_coverage",
        "centroid_lat_nn0_cell_coverage",
        "centroid_lon_nn1_cell_coverage",
        "centroid_lat_nn1_cell_coverage",
        "centroid_lon_nn2_cell_coverage",
        "centroid_lat_nn2_cell_coverage",
        "centroid_lon_nn3_cell_coverage",
        "centroid_lat_nn3_cell_coverage",
        "intersection_2_centroid_lon",
        "intersection_2_centroid_lat",
    )
    .to_pandas()
)

with open("data/folds/folds.pkl", "rb") as f:
    folds = pickle.load(f)

params = {
    "n_estimators": 212,
    "learning_rate": 0.002587,
    "max_depth": 9,
    "reg_alpha": 0.004842,
    "reg_lambda": 0.092771,
    "subsample": 0.807007,
    "colsample_bytree": 0.811025,
    "min_child_weight": 1,
    "grow_policy": "depthwise",
    "max_leaves": 0,
}


reg = MultiOutputRegressor(
    XGBRegressor(
        **params,
        sampling_method="uniform",
        tree_method="hist",
        n_jobs=-1,
        objective="reg:squarederror",
        seed=23,
    )
)

y = df[["Latitude", "Longitude"]].to_numpy()

X = df.drop(["Latitude", "Longitude", "cluster"], axis=1).to_numpy()

```

```
scorer = make_scorer(score_func=haversine_np, greater_is_better=False)

scores = cross_val_score(reg, X, y, scoring=scorer, cv=CustomCV(folds, df["cluster"]))

print(f"Mean = {scores.mean()} | Stddev = {scores.std()}\n")

print("Scores:\n", scores)

reg.fit(X, y)

feature_names = df.drop(["Latitude", "Longitude", "cluster"], axis=1).columns
for i, target_name in enumerate(["Latitude", "Longitude"]):
    model = reg.estimators_[i]
    importances = model.feature_importances_

    importance_df = pl.DataFrame(
        {"Feature": feature_names, "Importance": importances}
    ).sort("Importance", descending=True)

    print(f"\nTop 10 Feature Importance for {target_name}")
    print(importance_df.head(10))


y_pred = reg.predict(X)

plt.figure(figsize=(8, 6))
plt.scatter(y[:, 1], y[:, 0], c="blue", label="True", alpha=0.5, s=10)
plt.scatter(y_pred[:, 1], y_pred[:, 0], c="red", label="Predicted", alpha=0.5, s=10)
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend()
plt.title("True vs Predicted Locations")
plt.grid(True)
plt.tight_layout()
plt.show()
```

7 map_app.py

Purpose: Streamlit app to visualize coverage areas and data points spatially.

```

# Streamlit app displays a map of selected 5G cell coverages and user locations at ITU campus, allowing users to filter data by cell ID and neighbor range
import folium
import streamlit as st
from streamlit_folium import st_folium
import geopandas as gpd
import polars as pl

coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
df = pl.read_parquet("data/parquet/processed/dl.with_cluster.parquet").drop_nulls(
    ["Longitude", "Latitude"]
)  # null columns prevents marking user equipment locations on the map

gdf = gpd.GeoDataFrame(
    df.to_pandas(),
    geometry=gpd.points_from_xy(df["Longitude"], df["Latitude"]),
    crs="EPSG:4326",
    columns=df.columns,
)

st.set_page_config(layout="wide")
st.title("ITU Coverage Map")

col_1, col_2 = st.columns([5, 2])  # width ratio of columns on the app

m = folium.Map(location=[41.10427, 29.02554], zoom_start=15) # created starting location of the map manually

columns_to_check = [
```

```
    "NR_UE_PCI_0",
    "NR_UE_Nbr_PCI_0",
    "NR_UE_Nbr_PCI_1",
    "NR_UE_Nbr_PCI_2",
    "NR_UE_Nbr_PCI_3",
]

with col_2:
    selected_pci_list = st.multiselect("Choose Cell Id:", coverages["PCI"])

    start_n, end_n = st.select_slider(
        "Select the range of neighbor cells to include",
        options=[0, 1, 2, 3, 4],
        value=(0, 0),
    )

coverages = coverages[coverages["PCI"].isin(selected_pci_list)]

mask = False

for col in columns_to_check[start_n : end_n + 1]:
    mask |= gdf[col].isin(selected_pci_list)

gdf = gdf[mask]

folium.GeoJson(
    coverages.clipped_coverage,
    style_function=lambda x: {
        "fillColor": "red",
        "color": "black",
        "weight": 1,
        "fillOpacity": 0.2,
    },
).add_to(m)

folium.GeoJson(
    gdf,
    marker=folium.Circle(
        radius=10,
        fill=True,
        fill_color="blue",
        fill_opacity=0.2,
        color="black",
        weight=1,
    ),
).add_to(m)

with col_1:
    st_folium(m, height=500, use_container_width=True)
```

8 coverage_intersections.py

Purpose: Calculates coverage areas and their intersections.

```

from itertools import combinations
from shapely.ops import unary_union
import geopandas as gpd

coverages = gpd.read_parquet("data/parquet/coverages/coverages.parquet")
coverages["clipped_coverage"] = coverages.clipped_coverage.to_crs("EPSG:32635")

intersections_2 = []

for cell_1, cell_2 in combinations(coverages["PCI"], 2):
    coverage_poly_1 = coverages[coverages["PCI"] == cell_1].clipped_coverage.iloc[0]
    coverage_poly_2 = coverages[coverages["PCI"] == cell_2].clipped_coverage.iloc[0]

    intersection = coverage_poly_1.intersection(coverage_poly_2)

```

```
    if intersection.is_empty:
        convex_hull_flag = 1
        combined = unary_union([coverage_poly_1, coverage_poly_2])
        convex_hull = combined.convex_hull
        area = convex_hull.area
        shape = convex_hull

    else:
        convex_hull_flag = 0
        area = intersection.area
        shape = intersection

    intersections_2.append(
        {
            "sorted_cell_pairs": sorted([cell_1,cell_2]),
            "convex_hull": convex_hull_flag,
            "area_m2": area,
            "geometry": shape,
        }
    )

intersections_2 = gpd.GeoDataFrame(intersections_2, geometry="geometry", crs="EPSG:32635")
intersections_2["centroid"] = intersections_2.geometry.centroid.to_crs("EPSG:4326")
intersections_2 = intersections_2.to_crs("EPSG:4326")

intersections_2.to_parquet("data/parquet/coverages/coverages_intersect_2.parquet")
```